

```

1  import uart_tb_pkg::*;
2  interface uart_if(input logic clk);
3      logic reset;
4      logic rx;
5      logic [7:0] dout;
6      logic data_valid;
7  endinterface
8  module tb_uart_rx;
9      logic clk = 0;
10     always #5 clk = ~clk;
11     wire tick;
12
13     uart_if uif(clk);
14     baud_gen tick_gen
15     (
16         .clk(clk), .reset(uif.reset),
17         .dvsrc(10'd26),
18         .tick(tick)
19     );
20     uart_rx #(.DBIT(8),.SB_TICK(16)) DUT
21     (
22         .clk(clk),
23         .reset(uif.reset),
24         .rx(uif.rx),
25         .s_tick(tick),
26         .rx_done_tick(uif.data_valid),
27         .dout(uif.dout)
28     );
29
30     //uart_txn uart_txn;
31
32     mailbox #(uart_txn) gen2drv = new();
33     mailbox #(uart_txn) mon2sb = new();
34     mailbox #(uart_txn) gen2sb = new();
35
36     generator gen;
37     driver drv;
38     monitor mon;
39     scoreboard sb;
40
41     initial begin
42         gen = new(gen2drv);
43         drv = new(uif, gen2drv);
44         mon = new(uif, mon2sb);
45         sb = new(gen2sb, mon2sb);
46         uif.reset = 1;
47         uif.rx = 1; // idle
48         repeat (2) @(posedge clk);
49         uif.reset = 0;
50         fork
51             gen.run(10);
52             drv.run(16);
53             mon.run();
54             sb.run();
55         join_any
56         # 2000 $finish;
57     end
58 endmodule
59
60
61 module baud_gen
62 (
63     input logic clk, reset,
64     input logic [10:0] dvsrc,
65     output logic tick
66 );
67 // declaration
68 logic [10:0] r_reg;

```

```

70     logic [10:0] r_next;
71     // register
72     always_ff @(posedge clk, posedge reset)
73     begin
74         if(reset)
75             r_reg <= 0;
76         else
77             r_reg <= r_next;
78         end
79     // next_state logic
80     assign r_next = (r_reg == dvsr) ? 0 : r_reg + 1;
81     // output logic
82     assign tick = (r_reg == 1);
83 endmodule
84
85 module uart_rx
86     #(
87         parameter DBIT = 8,           // Dat
88         SB_TICK = 16
89     )
90     (
91         input logic clk, reset,
92         input logic rx, s_tick,
93         output logic rx_done_tick,
94         output logic [7:0] dout
95     );
96     // FSM state type
97     typedef enum{idle, start, data, stop} state_type;
98     // signal declaration
99     state_type state_reg, state_next;
100    logic [3:0] s_reg, s_next;
101    logic [2:0] n_reg, n_next;
102    logic [7:0] b_reg, b_next;
103    //FSMD state
104    always_ff @(posedge clk, posedge reset)
105    begin
106        if (reset) begin
107            state_reg <= idle;
108            s_reg <= 0;
109            n_reg <= 0;
110            b_reg <= 0;
111        end
112        else begin
113            state_reg <= state_next;
114            s_reg <= s_next;
115            n_reg <= n_next;
116            b_reg <= b_next;
117        end
118    end
119    always_comb
120    begin
121        state_next = state_reg;
122        rx_done_tick = 1'b0;
123        s_next = s_reg;
124        n_next = n_reg;
125        b_next = b_reg;
126        case (state_reg)
127            idle:
128                if (~rx) begin
129                    state_next = start;
130                    s_next = 0;
131                end
132                else
133                    state_next = idle;
134            start:
135                if(s_tick)
136                    if (s_reg == 7) begin
137                        state_next = data;
138                        s_next = 0;

```

```

139         n_next = 0;
140     end
141     else
142         s_next = s_reg + 1;
143 data:
144     if (s_tick)
145         if (s_reg == 15) begin
146             s_next = 0;
147             b_next = {rx, b_reg[7:1]};
148             if (n_reg == (DBIT - 1))
149                 state_next = stop;
150             else
151                 n_next = n_reg + 1;
152             end
153         else
154             s_next = s_reg + 1;
155 stop:
156         if (s_tick)
157             if (s_reg == (SB_TICK - 1)) begin
158                 state_next = idle;
159                 rx_done_tick = 1'b1;
160             end
161             else
162                 s_next = s_reg + 1;
163         endcase
164     end
165     // output
166     assign dout = b_reg;
167 endmodule
168

```